Kubernetes Cluster Deployment with kubeadm

Result: Two-node Kubernetes cluster (1 control plane, 1 worker).

1. Overview

The task was to build a multi-node Kubernetes cluster with `kubeadm`. The cluster was deployed on two Ubuntu virtual machines in VMware Workstation.

2. Environment

Item	Value
Hypervisor	VMware Workstation (NAT network)
Guest OS	Ubuntu (both nodes)
Control plane	`master-node`, `192.168.139.131`
Worker	`worker-1`, `192.168.139.128`
Container runtime	containerd
Kubernetes	v1.31 (installed: v1.31.14)
CNI	Calico v3.27.0
Pod network CIDR	`10.244.0.0/16`
Memory	Master 3 GiB, Worker 2 GiB

kubeadm requires at least 2 vCPUs on the control plane node and 1 cpu on worker.

3. VM Preparation

3.1 Cloning

1. Shut down the base Ubunt VM.

2. In VMware: right-click the VM, **Manage**, **Clone**, and select **Linked Clone** (stores only changes relative to the base VM).
3. Name the clones `k8s-master` and `k8s-worker1`.

3.2 Items that must be unique per clone

Cloning copies identifiers that must differ between nodes:

- **Hostname:** identical hostnames cause node registration conflicts.
- **MAC address:** kubeadm requires it to be unique.
- **machine-id:** duplicates can cause the DHCP server to issue the same IP to both machines.

MAC address. With the VM powered off: right-click the VM, **Settings**, **Network Adapter**, **Advanced**, **Generate**.

Hostname.

```
# On the control plane
sudo hostnamectl set-hostname master-node

# On the worker
sudo hostnamectl set-hostname worker-1
```

Open a new terminal session for the change to appear.

machine-id (on the cloned worker):

```
sudo rm -f /etc/machine-id /var/lib/dbus/machine-id
sudo systemd-machine-id-setup
sudo reboot
```

3.3 Issue: VMware SVGA error on VM start

Starting the worker VM produced:

```
VMware Workstation unrecoverable error: (svga)
Exception 0xc0000005
```

This is a virtual display adapter issue, not a network or MAC address issue. Resolution: **Settings**, **Display**, clear **Accelerate 3D graphics**, then start the VM. The same setting was disabled on the other VMs since 3D acceleration is not needed for server workloads.

3.4 Network verification

On the control plane:

```
ip a
ping 192.168.139.131
ping 192.168.139.128
```

The VMware NAT network uses the `192.168.x.x` range. The Calico default Pod CIDR (`192.168.0.0/16`) would overlap with the node network, so `10.244.0.0/16` was used instead.

4. Common Steps (both nodes)

4.1 Disable swap

The kubelet does not run with swap enabled.

```
sudo swapoff -a
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab #to permenantly disable
```

The `sed` command did not disable swap persistently on this system. See section 8.

4.2 Kernel modules and sysctl settings

```
sudo cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

sudo cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
```

Setting	Purpose
`overlay`	Overlay filesystem used by the container runtime for image layers
`br_netfilter`	Lets iptables process traffic crossing the Linux bridge used for Pod networking
`net.ipv4.ip_forward = 1`	Enables packet forwarding between network interfaces
`bridge-nf-call-iptables = 1`	Passes bridged traffic through iptables, which kube-proxy relies on for Services

Files in `/etc/modules-load.d` and `/etc/sysctl.d` persist across reboots; `modprobe` and `sysctl --system` apply the settings immediately.

4.3 Install containerd

```
sudo apt-get update
sudo apt-get install -y containerd

sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g'
/etc/containerd/config.toml

sudo systemctl restart containerd
sudo systemctl enable containerd
```

`SystemdCgroup = true` aligns containerd with the kubelet's cgroup driver. Verify with `sudo systemctl status containerd` (expected: `active (running)`).

4.4 Install kubeadm, kubelet, kubectl (v1.31)

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p -m 755 /etc/apt/keyrings

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | sudo gpg
--dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
```

v1.31 was used instead of v1.30, which is out of support. The `mkdir` of `/etc/apt/keyrings` is required on Ubuntu releases where the directory does not exist. `apt-mark hold` prevents unintended upgrades.

Verification:

```
kubeadm version
```

Output reported `GitVersion:"v1.31.14"`, `linux/amd64`.

4.5 Install conntrack

Required by kube-proxy and checked by kubeadm preflight (see section 5.1). Install on both nodes:

```
sudo apt-get update
sudo apt-get install -y conntrack
```

5. Control Plane Setup (master-node only)

5.1 Issue: preflight failure on kubeadm init

The first `kubeadm init` failed with:

```
[ERROR FileExisting-conntrack]: conntrack not found in system path
```

A `[WARNING Swap]` message was also printed. Resolution:

```
sudo apt-get install -y conntrack
sudo swapoff -a
```

5.2 Initialize the cluster

```
sudo kubeadm init --apiserver-advertise-address=192.168.139.131 --pod-network-cidr=10.244.0.0/16
```

- `--apiserver-advertise-address`: IP on which the API server advertises itself (the control plane's IP).
- `--pod-network-cidr`: address range for Pods.

kubeadm logged `remote version is much newer: v1.37.1; falling back to: stable-1.31` and used `v1.31.14`. Initialization completed with `Your Kubernetes control-plane has initialized successfully!` and printed the `kubeadm join` command. If the command is lost, regenerate it:

```
kubeadm token create --print-join-command
```

kubeadm deployed the API server, scheduler and controller manager as static pods, plus etcd, kube-proxy and CoreDNS.

5.3 Configure kubectl

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

5.4 Install Calico

Kubernetes has no built-in Pod network, so a CNI plugin is required. The manifest was downloaded and its pool CIDR modified to match `--pod-network-cidr` before applying:

```
curl -O
https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/cali
co.yaml

sed -i 's/# - name: CALICO_IPV4POOL_CIDR/- name: CALICO_IPV4POOL_CIDR/g'
calico.yaml
sed -i 's/# value: "192.168.0.0\/16"/  value: "10.244.0.0\/16"/g' calico.yaml

kubectl apply -f calico.yaml
```

This created the `calico-node` DaemonSet and the `calico-kube-controllers` Deployment.

5.5 Monitoring startup

```
kubectl get nodes -w
kubectl get namespaces
kubectl get pods -n kube-system
```

- The control plane showed `NotReady` until Calico started, then became `Ready` at about 13 minutes of cluster age.
- Namespaces present: `default`, `kube-node-lease`, `kube-public`, `kube-system`.
- In `kube-system`, `etcd`, `kube-apiserver`, `kube-controller-manager`, `kube-scheduler` and `kube-proxy` were `Running`. `calico-node` was at `Init:2/3`, and `calico-kube-controllers` and both CoreDNS pods were `ContainerCreating` while images were pulled.

```
daemonset.apps/calico-node created
deployment.apps/calico-kube-controllers created
mazen@master-node:~$ kubectl get nodes -w
NAME          STATUS     ROLES           AGE     VERSION
master-node   NotReady   control-plane   7m49s   v1.31.14
master-node   NotReady   control-plane   10m     v1.31.14
^Cmazen@master-node:~$ kubectl get nodes
NAME          STATUS     ROLES           AGE   VERSION
master-node   NotReady   control-plane   12m   v1.31.14
mazen@master-node:~$ kubectl get nodes -w
NAME          STATUS     ROLES           AGE   VERSION
master-node   NotReady   control-plane   12m   v1.31.14
master-node   Ready      control-plane   13m   v1.31.14
master-node   Ready      control-plane   13m   v1.31.14
^Cmazen@master-node:~$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   13m
kube-node-lease   Active   13m
kube-public       Active   13m
kube-system       Active   13m
mazen@master-node:~$ kubectl get pods -n kube-system
NAME                                  READY   STATUS              RESTARTS   AGE
calico-kube-controllers-868cbf9cc-8ljd5   0/1     ContainerCreating   0          6m21s
calico-node-p2hrg                     0/1     Init:2/3            0          6m21s
coredns-7c65d6cfc9-8w5k4              0/1     ContainerCreating   0          13m
coredns-7c65d6cfc9-l8c9w              0/1     ContainerCreating   0          13m
etcd-master-node                      1/1     Running             0          14m
kube-apiserver-master-node            1/1     Running             0          14m
kube-controller-manager-master-node   1/1     Running             0          14m
kube-proxy-kqjbm                      1/1     Running             0          13m
kube-scheduler-master-node            1/1     Running             0          14m
mazen@master-node:~$
```

Figure 1: Node status and kube-system pods during Calico startup.

6. Worker Setup (worker-1 only)

6.1 Issue: kubelet not healthy during join

The join then failed after the 4-minute kubelet health check:

```
[kubelet-check] The kubelet is not healthy after 4m0.003405558s
error execution phase kubelet-start: ... context deadline exceeded
```

A `[WARNING Swap]` message was present, and swap was the most likely cause (the kubelet does not start with swap enabled).

```
# 1. Clean up the failed attempt
sudo kubeadm reset -f

# 2. Disable swap and confirm
sudo swapoff -a
free -h

# 3. Re-apply containerd configuration
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g'
/etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl status containerd

# 4. Retry the join
sudo kubeadm join 192.168.139.131:6443 --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH>
```

`<TOKEN>` and `<HASH>` come from the `kubeadm init` output. Tokens expire after 24 hours.

6.2 Successful join

```
This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was
received.
* The Kubelet was informed of the new secure connection details.
```

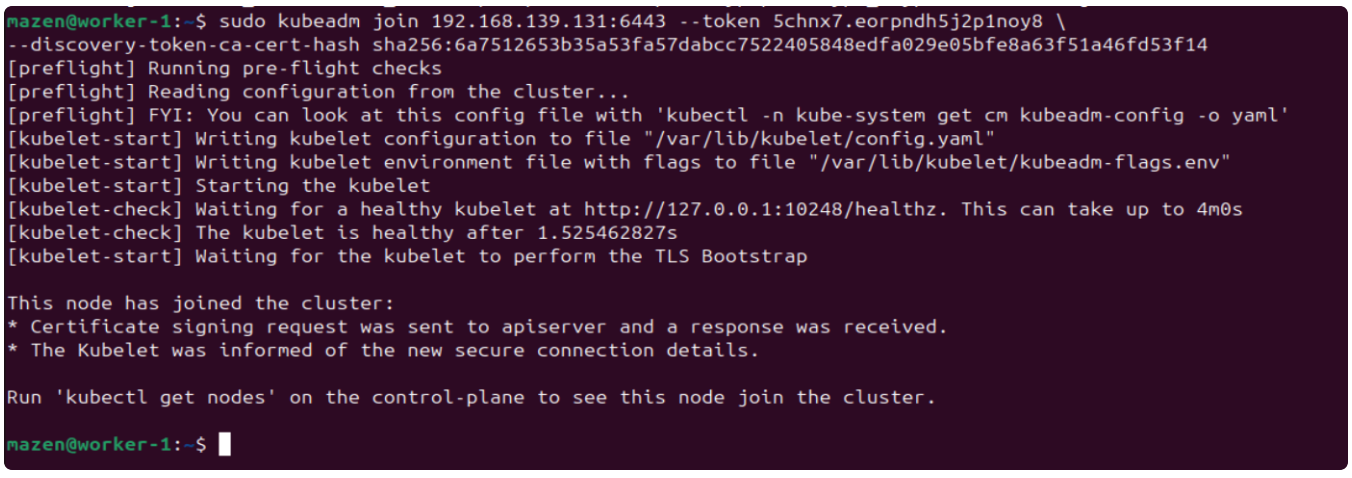


Figure 2: `kubeadm join` completing on worker-1.

6.3 kubectl on the worker

Running `kubectl get pods` on the worker returned `The connection to the server localhost:8080 was refused`. `/etc/kubernetes/admin.conf` exists only on the control plane (`cp: cannot stat '/etc/kubernetes/admin.conf'` on the worker). All kubectl commands are run from the control plane. If kubectl is needed on the worker, copy the kubeconfig from the master with `scp`.

7. Issue: API Server Unreachable After Control Plane Restart

After the control plane VM hung and was restarted, kubectl returned:

```
The connection to the server 192.168.139.131:6443 was refused - did you
specify the right host or port?
```

`free -h` showed swap (3.7 GiB) active again after the reboot. Resolution:

```
sudo swapoff -a
sudo systemctl restart containerd
sudo systemctl restart kubelet
# wait about one minute
kubectl get nodes
```

8. Making swap-off persistent

Swap returned after reboot because the `sed` command in section 4.1 did not comment out the swap entry in `/etc/fstab` (on this Ubuntu release the entry is `/swap.img`). Edit the file manually on both nodes:

```
sudo nano /etc/fstab
```

Comment out the swap line:

```
#/swap.img  none  swap  sw  0  0
```

9. Final Verification

From the control plane:

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master-node	Ready	control-plane	37m	v1.31.14
worker-1	Ready	<none>	8m4s	v1.31.14

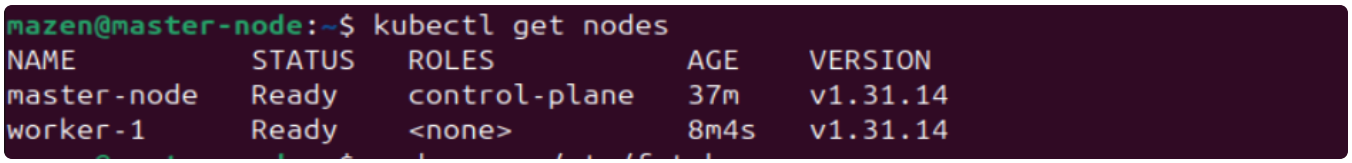


Figure 3: `kubectl get nodes` showing both nodes `Ready`.

10. Issue Summary

#	Issue	Cause	Resolution
1	VMware `svga` Exception 0xc0000005 on VM start	3D graphics acceleration	Disable Accelerate 3D graphics in Display settings
2	`ERROR FileExisting-conntrack` on `kubeadm init`	`conntrack` not installed	`sudo apt-get install -y conntrack` on both nodes
3	`ERROR IsPrivilegedUser` on join	Command run without root	Run with `sudo`
4	`kubelet is not healthy after 4m0s`	Swap enabled (likely cause)	`kubeadm reset -f`, `swapoff -a`, verify containerd, retry join
5	API server refused connections on 6443 after reboot	Swap re-enabled, kubelet not running	`swapoff -a`, restart containerd and kubelet
6	Swap re-enabled on every reboot	`/swap.img` entry not commented out in `/etc/fstab`	Comment out the entry manually

#	Issue	Cause	Resolution
7	`localhost:8080` refused on worker	kubeconfig exists only on control plane	Run kubectl from the control plane

11. Procedure Checklist

1. Clone the VMs; generate a new MAC address, set hostnames, regenerate machine-id.
2. Verify IP addresses and connectivity (`ip a` , `ping`).
3. Both nodes: disable swap, including the `/etc/fstab` entry.
4. Both nodes: load `overlay` and `br_netfilter` , apply sysctl settings.
5. Both nodes: install and configure containerd (`SystemdCgroup = true`).
6. Both nodes: install kubeadm, kubelet, kubectl v1.31 and hold the packages; install `conntrack` .
7. Control plane: `kubeadm init` with `--apiserver-advertise-address` and `--pod-network-cidr=10.244.0.0/16` .
8. Control plane: copy `admin.conf` to `~/.kube/config` .
9. Control plane: modify the Calico manifest CIDR and apply it.
10. Worker: `sudo kubeadm join` with the token and discovery hash.
11. Control plane: `kubectl get nodes` and confirm both nodes are `Ready` .